Darshan UNIVERSITY

योग: कर्मसु कौशलम्

Python Programming - 2301CS404

Lab - 12

Roll No : 236

Name : AASTHA KATAKIA

Exception Handling

01) WAP to handle following exceptions:

1. ZeroDivisionError
2. ValueError
3. TypeError

Note: handle them using separate except blocks and also using single except block too.

In [6]:
```
try:
    a = int(input("Enter a:"))
    b = input("Enter b:")

    result = a+b
    print("Result:",result)

except(ZeroDivisionError,ValueError,TypeError) as e:
    print("Error occured",e)

finally:
    print("Program terminated!")
```

```
Error occured unsupported operand type(s) for +: 'int' and 'str'
Program terminated!
```

02) WAP to handle following exceptions:

1. IndexError
2. KeyError

In [8]:
```python
try:
    my_list = [10,20,30]

    index = int(input("Enter index:"))
    print("Value at index:",my_list[index])

    my_dict = {"name": "Aastha", "age": 21}
    key = input("Enter key : ")
    print("Value of key:", my_dict[key])

except (IndexError,KeyError) as e:
    print("Error occurred",e)

finally:
    print("Program terminated!")
```

```
Value at index: 30
Value of key: 21
Program terminated!
```

03) WAP to handle following exceptions:

1. FileNotFoundError
2. ModuleNotFoundError

In [11]:
```python
try:
    filename = input("Enter file name: ")
    file = open(filename, "r")
    print(file.read())
    file.close()

    import abc

except (FileNotFoundError, ModuleNotFoundError) as e:
    print("Error occurred:", e)

finally:
    print("Program Terminated")
```

```
Error occurred: [Errno 2] No such file or directory: 'akkad bakkad bambe bo'
Program Terminated
```

04) WAP that handles all type of exceptions in a single except block in standard error message format.

In [13]:
```
try:
    a = int(input("Enter a: "))
    b = int(input("Enter b: "))

    result = a / b
    print("Result:", result)

    my_list = [10, 20, 30]
    print("Enter index:", my_list[5])

    my_dict = {"name": "ABC"}
    print(" Enter key:", my_dict["age"])

    file = open("sample.txt", "r")

    import abc

except Exception as e:
    print("Standard Error Message:", e)

finally:
    print("Program terminated.")
```

```
Result: 0.6666666666666666
Standard Error Message: list index out of range
Program terminated.
```

05) WAP to demonstrate else and finally block.

In [1]:
```
try:
    a = int(input("Enter a: "))
    b = int(input("Enter b: "))

    result = a / b
    print("Result :", result)

except ZeroDivisionError:
    print("Error: Cannot divide by zero!")

except ValueError:
    print("Error: Please enter valid number!")

else:
    print("Division successful.")

finally:
    print("Program terminated!")
```

```
Result : 2.0
Division successful.
Program terminated!
```

06) WAP to create an udf divide(a,b) that handles ZeroDivisionError.

In [4]:
```
def divide(a, b):
    try:
        result = a / b
        return result
    except ZeroDivisionError:
         print("Error: Cannot divide by zero!")
         return None

x = int(input("Enter x: "))
y = int(input("Enter y: "))

output = divide(x, y)

if output is not None:
    print("Result :", output)
```

```
Error: Cannot divide by zero!
```

07) WAP to accept item prices and calculate total. Raise ValueError if price is negative, otherwise print the total bill amount.

In [4]:
```
try:
    n = int(input("Enter number of items: "))
    total = 0

    for i in range(n):
        price = float(input(f"Enter price of item {i+1}: "))

        if price < 0:
            raise ValueError("Price cannot be negative!")

        total += price

    print("Total bill amount =", total)

except ValueError as e:
    print("Error:", e)
```

```
Total bill amount = 50.0
```

08) Create a short program that prompts the user for a list of grades separated by commas.

Split the string into individual grades and use a list comprehension to convert each string to an integer.

You should use a try statement to inform the user when the values they entered cannot be converted.

In [7]:
```
try:
    grades_input = input("Enter grades separated by commas: ")
```

```
    grades = [int(g.strip()) for g in grades_input.split(",")]

    print("Grades list:", grades)

except ValueError:
    print("Error: Please enter only numbers separated by commas.")
```

```
Grades list: [90, 80, 70]
```

09) Accept 5 subject marks (0-100). Use assert for validation.

Sample Input (Error Case)

Enter marks: 110

Sample Output (Error Case)

AssertionError: Marks should be between 0 and 100.

Sample Input (Normal Case)

Valid marks entered

Sample Output (Normal Case)

Average Marks displayed

In [9]:
```
marks = []

try:
    for i in range(5):
        m = int(input("Enter marks: "))

        # validation using assert
        assert 0 <= m <= 100, "Marks should be between 0 and 100."

        marks.append(m)

    print("Valid marks entered")

    avg = sum(marks) / len(marks)
    print("Average Marks =", avg)

except AssertionError as e:
    print("AssertionError:", e)
```

```
Valid marks entered
Average Marks = 76.0
```

10) WAP that gets password from user. Password must have at least 8 characters and one digit.

Raise WeakPasswordError if invalid, print the messsage "Correct" otherwise.

In [11]:
```
class WeakPasswordError(Exception):
    pass

try:
    password = input("Enter password: ")


    if len(password) < 8:
        raise WeakPasswordError("Password must contain at least 8 characters.")

    if not any(ch.isdigit() for ch in password):
        raise WeakPasswordError("Password must contain at least one digit.")

    print("Correct")

except WeakPasswordError as e:
    print("WeakPasswordError:", e)
```

```
Correct
```

11) WAP to raise your custom Exception named NegativeNumberError with the error message : "Cannot calculate the square root of a negative number" :

if the given number is negative.

otherwise print the square root of the given number.

In [13]:
```
import math


class NegativeNumberError(Exception):
    pass

try:
    num = float(input("Enter a number: "))

    if num < 0:
        raise NegativeNumberError(
            "Cannot calculate the square root of a negative number"
        )

    result = math.sqrt(num)
    print("Square root =", result)

except NegativeNumberError as e:
    print("NegativeNumberError:", e)
```

```
Square root = 6.708203932499369
```

In [ ]: